

识破聚合假象

探究数据的真实分布

模块五: 数据工程 (下)

第十八讲: 识破聚合假象——探究数据的真实分布

【解释】

本讲定位

上节课学的是用条形图做总量排名。这节课要解决的问题是：总量排名会骗人。一个省的景点总数排第一，不代表它的景点质量也排第一。我们将引入 Seaborn 库，用散点图和箱线图来拆解聚合数字背后的质量差异和异常值。这是从“会画图”到“会用图发现问题”的关键一步。

一、总量统计的表象与误导

为什么总排名不能代表一切？

[解释]

为什么要质疑总量

上节课学了 Groupby 和条形图之后，很多人会觉得"排名画出来了，任务就完成了"。这一章要打破这个认知：排名只说明了"谁多"，没有说明"多的那些质量怎么样"。这是后面引入分布图的逻辑起点。

聚合数据的遮蔽效应

条形图能快速展示哪个组别的体量大，但它无法反映组内个体之间的质量差异。

均值与聚合的陷阱

- **总量误导**：某个地区的景点总数排名第一，但这个数字里可能包含了大量长期闲置的免费低质景点。
- **平均数失真**：某区域景点的门票均价远低于全国水平，未必代表惠民——更可能是因为名册里存在大量零收费的街心公园，把真正收费景点的价格拉低了。

怎么看到真实面貌？

如果想了解一个群体的真实情况，不能只算总数或平均值。需要把每一条记录单独投射到坐标轴上，观察它们的离散程度和密集区域。这就是"分布分析"的基本思路。

💡 **统计常识**：一张包含马云和五百个普通员工的报表，算出来的平均家庭资产没有参考意义。在学术和工程分析中，呈现分布情况比计算一个总分重要得多。

[解释]

平均数为什么会骗人

这一页讲的是一个基础但容易被忽视的统计常识：平均值会被极端值严重干扰。如果一个地区的旅游数据里混入了几百个免费公园，这些零值会把少数高价景区的票价信号完全稀释掉。所以，分析不能停留在"加总求平均"这一步，必须看每条记录的分布情况。

用代码验证：聚合骗局的真实面目

与其空谈"聚合会误导"，不如用代码跑一遍，亲眼看看总量冠军的真实内幕。

第一步：总量排名

```
# L17 学过的 groupby — 按省份统计景区数量
top = df.groupby('province')['name'].count() \
        .sort_values(ascending=False).head(5)
print(top)
```

广东	11584
浙江	10414
江苏	7928
四川	7066
山东	6461

广东以 11584 家景区遥遥领先。看起来广东旅游资源最丰富。

第二步：拆开看票价构成

```
# 广东省免费景区占多少？
gd = df[df['province'] == '广东']
free = gd[gd['price'] == 0]
print(f'广东总计 {len(gd)} 家')
print(f'其中免费 {len(free)} 家')
print(f'免费占比 {len(free)/len(gd)*100:.0f}%')
```

广东总计	11584 家
其中免费	9340 家
免费占比	81%

真相：广东超过八成是免费景点，靠数量堆上了排名第一。

这就是聚合假象： 总量冠军 $\neq$ 质量冠军。不拆开看每条记录的构成，仅凭加总数字做判断，结论很可能是错的。这正是我们需要分布分析工具的原因。

[解释]

用代码验证聚合骗局

口说“排名会骗人”不如看数据——代码输出告诉你，广东 11584 家景区中 81% 是免费的。这种数据冲击比文字描述更有说服力。

左侧代码复用了 L17 学过的 groupby，右侧用了上周学的 DataFrame 条件筛选。两段代码合在一起构成了一个完整的“验证→拆解→发现”链条，这就是为什么后面要引入分布分析工具。

二、引入 Seaborn 绘图库

专门用于分布分析的可视化工具

[解释]

为什么需要新工具

上节课用的 Matplotlib 擅长画条形图、折线图这类基础图表。但如果要把几千条记录按星级分组、用不同颜色标注、再散布到坐标轴上，用 Matplotlib 需要写大量的循环和配色代码。Seaborn 把这些常见操作封装好了，一行代码就能完成。这就是引入它的原因。

Seaborn 与 Matplotlib 的关系

用 Matplotlib 给几千个景区按星级配色并画成散点图，需要手写多层循环。Seaborn 把这类操作简化成了一行函数调用。

Seaborn 的定位

- Seaborn (`sns`) 是建立在 Matplotlib 之上的统计图表库。它内置了配色方案和统计计算，适合画散点图、箱线图、热力图等分布类图表。
- 它直接接受 Pandas 的 DataFrame 作为输入，指定列名即可完成分组和绑定，不需要手动提取数据再传入。

环境配置

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

df = pd.read_csv('qunar_full.csv')

# 启用 Seaborn 内置的网格主题
sns.set_theme(style='whitegrid')

# 仍然需要配置 Matplotlib 的中文字体
plt.rcParams['font.sans-serif'] = ['Arial Unicode MS', 'SimHei']
plt.rcParams['axes.unicode_minus'] = False
```

注意： Seaborn 的底层渲染仍然依赖 Matplotlib。如果不配置中文字体，Seaborn 画出的图表标题和坐标轴会显示为乱码方块。因此，两个库需要一起导入和配置。

[解释]

两个库的配合关系

初学者容易犯的一个错误是：以为装了 Seaborn 就不需要管 Matplotlib 了。实际情况是，Seaborn 负责高层的图表逻辑（分组、配色、统计计算），但最终把图画到屏幕上的工作还是由 Matplotlib 完成。所以中文字体、坐标轴格式这些底层设置，仍然要通过 `plt.rcParams` 来配置。两个库一起导入是标准做法。

直方图 (histplot): 从条形图到分布图的桥梁

L17 的条形图回答的是"哪个省景区最多"。直方图回答的是另一类问题: "票价集中在哪个区间? 贵的多还是便宜的多?"

条形图 vs 直方图

对比维度	条形图 (bar)	直方图 (hist)
X 轴	类别 (省份)	连续数值区间
Y 轴	计数	频率/频次
回答问题	谁最多	分布形状
柱子间	有间距	紧密相连

代码示例

```
# 画全国景区的票价频率分布
sns.histplot(data=df[df['price'] > 0],
             x='price',
             bins=30,
             kde=True)
plt.title('全国景区票价分布')
plt.xlabel('票价 (元)')
plt.ylabel('频次')
plt.show()
```

- bins=30 : 把价格范围分成 30 个区间
- kde=True : 叠加一条平滑的密度曲线

💡 直觉建立: 如果直方图呈"右偏"形状 (左高右低), 说明大多数景区票价低, 少数极贵。这时平均值会被极端高价拉高, **中位数才是更可靠的集中趋势指标**。

[解释]

直方图作为认知过渡

直方图在教学上的定位是条形图到箱线图之间的"桥梁"。条形图比较的是不同类别的数量, 直方图展示的是单一数值变量的频率分布形状。掌握了直方图的"右偏""左偏"概念后, 再看箱线图的 Q1/Q2/Q3 就更容易理解。右偏分布下"平均值 vs 中位数"的对比是一个重要的统计直觉。这直接呼应了 §一 "聚合假象"的主题——平均值之所以会骗人, 正是因为极端值的拉拽。课堂实践中也要求学生判断自己省份的票价直方图是否呈偏斜形状。

绘图前的数据过滤

要分析星级景区的价格分布，首先需要从数据集中排除没有星级和没有票价的记录，避免大量无效数据干扰图表。

过滤条件

- 排除 `level` 为"无"的记录（没有星级评定的景点）。
- 排除 `price` 为 0 的记录（免费或未收录票价的景点）。
- 将过滤后的结果存入新的 DataFrame 用于后续绑图。

过滤代码

```
# 只保留有星级、且有票价的景点
filtered_df = df[
    (df['level'] != '无') &
    (df['price'] > 0)
]

print(f"过滤完成，剩余有效记录：{len(filtered_df)} 条")
```

为什么要过滤：数据集中大量免费小公园的记录如果不剔除，散点图上会出现密集的黑色团块堆积在 origin 附近，把真正有分析价值的区域全部压缩掉。制图的目的是回答问题，不是展示数据量。

[解释]

为什么需要先过滤

这一步把上周学的 DataFrame 条件筛选和本节课的制图串联起来了。清洗和制图不是两件独立的事，而是前后衔接的流程。数据集里将近 85,000 条记录没有星级，17,000 多条票价为零。如果不过滤就直接画图，这些无关记录会在图表上形成大面积的密集区域，导致真正需要观察的价格分布被严重压缩，看不出任何规律。

三、散点图与箱线图

定位分布规律与异常值（Outliers）

[解释]

本章内容

这是本讲的核心实操章节。前面讲了为什么不能只看总量排名，这里给出具体的替代方案：用散点图观察个体分布，用箱线图对比不同组别的中位数和离散程度。两种图各有适用场景，后面会分别演示。

先画一张"原始散点图"看看

在开始分布分析之前，我们先不做任何额外清洗，直接把过滤后的数据画成散点图——看看会发生什么。

你看到了什么？

运行代码后，你大概率会看到一张 "生病的图"：

1. **压缩畸形**：绝大多数点挤在左下角一小块区域
2. **右侧飞点**：极少数点飘到了几千甚至上万的价格区
3. **顶部飞点**：个别景区销量高达数万，把 Y 轴拉扯变形

初步判断：这张图被 **离群点 (Outliers)** 劫持了。

代码

```
plt.figure(figsize=(10, 6))
sns.scatterplot(
    data=filtered_df,
    x='price', y='sale_count',
    alpha=0.3, s=10
)
plt.title('原始散点图（未清洗）')
plt.show()
```

离群点的危害：少数极端值会「劫持」坐标轴的刻度范围，把 95% 的正常数据压缩成一团看不清的黑块。这就是为什么专业分析师在画分布图之前，必须先诊断并处理离群点。

[解释]

离群点对图表的影响

离群点 (Outliers) 是指远离数据主体分布的极端数值。它们不一定是错误数据——可能是迪士尼乐园这种定价几千元的大型商业综合体，或者某个网红景区单月销量暴增。但在制图层面，即使只有一两个极端值，也会把坐标轴的范围拉大几十倍，导致其他数据被压缩到几乎看不见。这就是为什么分布分析的第一步不是直接画图，而是先做离群点诊断。

离群点诊断：用统计方法定位问题

三种常见离群点

类型	表现	原因
天价型	price > 1000	豪华套票、VIP 套餐
爆量型	sale_count > 10000	网红景区、平台刷单
零值型	price = 0 且有星级	数据缺失或免费活动

统计诊断

```
print('=== 离群点诊断 ===')
print(f'price > 1000: '
      f'{(filtered_df["price"]>1000).sum()} 条')
print(f'sale_count > 10000: '
      f'{(filtered_df["sale_count"]>10000).sum()} 条')
print(f'price 的 99 分位: '
      f'{filtered_df["price"].quantile(0.99):.0f} 元')
```

IQR 方法（箱线图的数学基础）

统计学家 John Tukey 定义的异常值标准：

- $IQR = Q3 - Q1$ （四分位距）
- 下界 = $Q1 - 1.5 \times IQR$
- 上界 = $Q3 + 1.5 \times IQR$
- 超出上下界的数据点即为统计离群点

这也是后面箱线图中那些"飞出须线的小圆点"的来源。

 **关键判断：**离群点不能盲目删除。天价景区虽然影响制图，但它确实存在。正确做法是根据分析目标设定合理的过滤边界，而不是一刀切。

[解释]

IQR 方法与 Tukey 准则

IQR（Inter-Quartile Range，四分位距）是 $Q3 - Q1$ 的距离，代表了数据中间 50% 的分布宽度。Tukey 在 1977 年提出用 1.5 倍 IQR 作为判定异常值的边界——超出 $Q3 + 1.5 \times IQR$ 或低于 $Q1 - 1.5 \times IQR$ 的数据点就是统计意义上的离群点。箱线图上那些画在须线（whisker）外面的小圆点，就是按照这个标准标记出来的。这个方法不依赖数据是否服从正态分布，对偏态数据同样适用，因此在探索性数据分析中被广泛使用。

清洗后的对比：真相显现

清洗策略

根据分析目标（普通收费景区的价格分布），设定合理边界：

```
# 清洗：去极端价格和极端销量
df_clean = filtered_df[
    (filtered_df['price'] <= 500) &
    (filtered_df['sale_count'] <= 10000)
]
print(f'清洗前: {len(filtered_df)} 条')
print(f'清洗后: {len(df_clean)} 条')
```

Before vs After

清洗前后对比效果：

- **清洗前**：图形严重畸形，95% 数据挤在左下角
- **清洗后**：数据分布清晰可见，能辨识出价格集中区间和星级差异

核心经验： 分布分析的正确流程不是"画图"，而是**"画图 → 诊断 → 清洗 → 再画图"**。第一张"生病的图"不是失败，而是发现问题的起点。

[解释]

为什么清洗后图形会"变好"

离群点被移除后，坐标轴的范围大幅缩小。原来 X 轴要伸展到几千元才能容纳那一两个天价记录，现在只需要伸展到 500 元。这意味着每个像素代表的价格范围变小了很多倍，图表的分辨率相应提升。数据密集区域的内部结构（比如不同星级的价格分层）自然就变得可以辨识了。这个原理在统计学文献中被称为"视觉分辨率"（Visual Resolution）——图表的有效信息量取决于坐标轴范围与数据实际分布范围的匹配程度。

散点图 (Scatterplot)

散点图将过滤后的每一条记录作为一个独立的点，投射到二维坐标系上。

三个变量同时呈现

Seaborn 的 `scatterplot` 函数支持通过 `hue` 参数把分类字段（如 `level`）映射为不同颜色。这样一张图就能同时展示三个维度的信息：X 轴（价格）、Y 轴（销量）和颜色（星级）。

代码示例

```
plt.figure(figsize=(10, 6))

# x: 价格, y: 销量, hue: 按星级着色
sns.scatterplot(
    data=filtered_df,
    x='price',
    y='sale_count',
    hue='level'
)

plt.title('星级景区票价与销量分布')
plt.show()
```

💡 **识别异常值 (Outliers):** 散点图上远离主体集群、孤立存在的点，可能是数据录入错误，也可能是具有特殊商业模式的个案。这些异常点是值得深入调查的线索，而总量统计永远不会暴露它们。

[解释]

散点图的作用

散点图把每条记录都画成一个独立的点。通过这种方式，可以直观地看到大多数景区的票价集中在哪个区间，销量集中在哪个量级。更重要的是，那些远离主体集群的孤立点会立刻暴露出来——比如某个景区票价标到几千块但销量为零，这在总量统计里完全看不出来，但在散点图上一眼就能发现。这些孤立点就是统计学里说的“异常值” (Outliers)。

箱线图 (Boxplot)

当数据量大、散点严重重叠时，散点图的可读性会下降。箱线图用统计摘要来展示分布特征，更适合做分组对比。

箱线图的结构

- **箱体**：代表数据的中间 50%（第 25 百分位到第 75 百分位）。箱体越窄，说明这个区间内的价格越集中。
- **须线与离群点**：超出箱体范围的数据用须线延伸，更远的极端值用独立的圆点标出。这些点就是平均数无法反映的异常样本。

代码示例

```
plt.figure(figsize=(10, 6))

# 横向箱线图: y 为分组变量, x 为数值变量
sns.boxplot(
    data=filtered_df,
    y='level',
    x='price',
    palette='Set2'
)

plt.title('各星级景区票价分布对比')
plt.show()
```

值得关注的现象：在箱线图中，可能会发现某些低星级（如 3A）景区的个别票价，远高于多数 5A 景区的中位数。这种“逆向定价”现象在总量排名中完全不可见。

[解释]

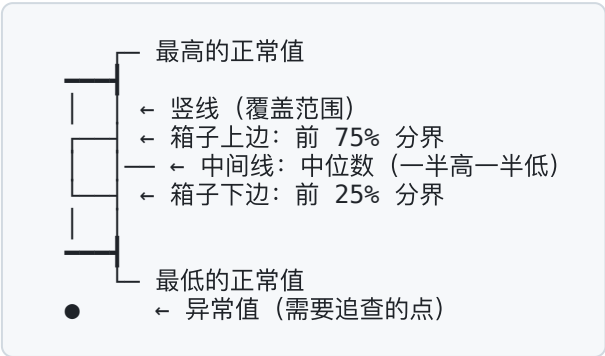
箱线图的读法

箱线图用五个统计量来概括一组数据的分布：最小值、第 25 百分位、中位数、第 75 百分位、最大值。箱体覆盖的是中间 50% 的数据，所以箱体越窄，说明这个星级内的票价越集中。超出须线范围的独立圆点就是异常值。箱线图的优势在于分组对比：把 5A、4A、3A 的箱体排在一起，每个星级的价格集中区间和离散程度一目了然。如果发现低星级组的异常点比高星级的中位数还高，这就是一个值得深入调查的信号。

读懂箱线图：只需记住 3 条规则

箱线图看起来复杂，但你不需要记公式。只要掌握 3 条读图规则，就能从中提取有价值的信息。

箱线图长什么样



箱子 = 中间 50% 的数据所在区间
公式不用记，记住这一句就够了。

3 条读图规则

你看到的现象	说明了什么
规则 1：箱子窄	这一组的数据很集中，差异小
规则 2：箱子宽	这一组的数据很分散，差距大
规则 3：有圆点	存在异常值 → 必须追问：是谁？为什么？

核心能力：看到圆点不要只说“有异常值”。要追问两个问题：

- 1. 这些异常值具体是哪些景区？
- 2. 为什么它们的票价这么特殊？

追问，是人比 AI 更有价值的地方。

[解释]

箱线图怎么读

这一页没有列 IQR 公式和须线范围计算，因为对大多数人来说，四分位距的数学定义不重要。你只需要记住三条规则：箱子 = 中间 50%，线条 = 正常范围，圆点 = 异常值。这三条足以支撑后面所有的读图。
底部那句“追问是人比 AI 更有价值的地方”值得记住——AI 能画图、能算统计量，但它不会主动追问“为什么数据有异常”。这个追问能力需要你自己来培养。

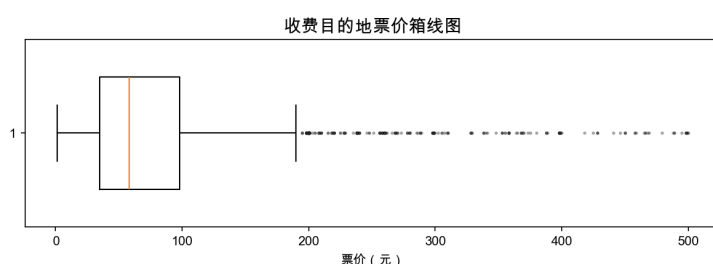
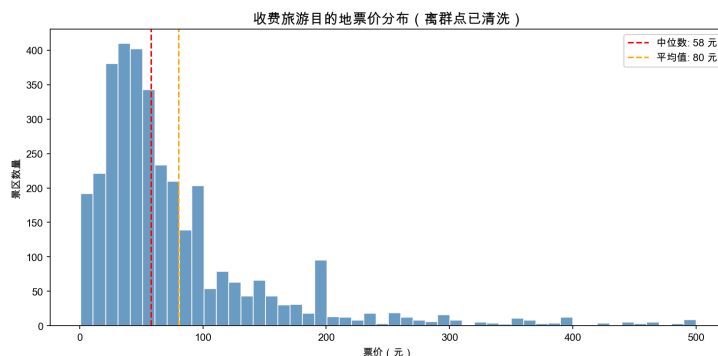
实操预期：票价分布分析

Notebook Part 1 预期结果

在 Notebook 中完成 Part 1，你应该看到两张图：直方图显示票价的集中区间，箱线图凸显异常值。

参考 Prompt

请画一张直方图，展示 price 列的频率分布，bins=50，用 steelblue 颜色。标注中位数（红色虚线）和均值（橙色虚线）。



🔪 思考题 1

1. 平均值和中位数哪个更大？这说明分布是左偏还是右偏？
2. 箱线图右侧的异常值点是否超出了 Part 0.5 的 Tukey 上界？
3. 如果不做离群点清洗，直方图会是什么形状？

[解释]

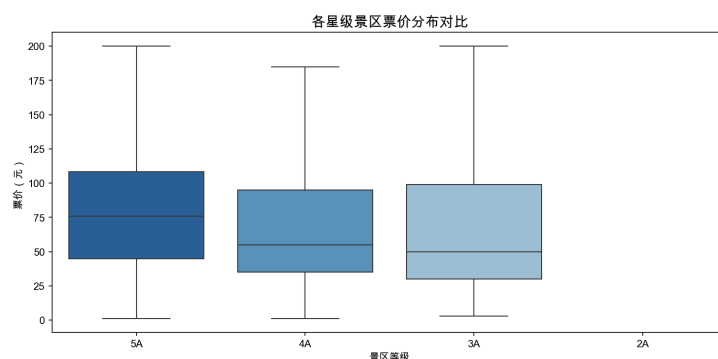
为什么同一份数据要用两种图看

直方图和箱线图各有专长。直方图擅长展示分布的整体形状（是右偏还是对称、有没有双峰），但看不出精确的中位数和四分位数。箱线图刚好相反：它用五个统计量精确概括数据，但看不出分布的具体形状。所以在探索性分析中，两者配合使用是标准做法。思考题第 3 题让你回想 Part 0.5 的清洗——如果不做离群点处理，直方图会被极端值严重压缩，看不出任何规律。

实操预期：各星级票价分布对比

Notebook Part 2 预期结果

分组箱线图直接回答："5A真的比 3A 贵吗？贵多少？"



参考 Prompt

用 seaborn 的 boxplot 画一张分组箱线图。
X 轴为 level (5A/4A/3A/2A),
Y 轴为 price。
配色用 Blues_r, 隐藏异常值点。

🔑 思考题 2

1. 5A 景区的中位数票价大约是多少？和 3A 的差距多大？
2. 哪个星级的票价离散度（箱体宽度）最大？为什么？
3. 这张图对应可视化七层分析中的哪个层次？

[解释]

为什么这里隐藏了异常值点

你可能注意到 Prompt 中写了“隐藏异常值点”。这不是偷懒，而是有目的的取舍：当你想对比不同星级的箱体位置时，异常值点会让图变得很乱，干扰你对箱体高低的判断。画图时每个参数都是一个分析决策——“要不要显示异常值”取决于你这张图的目的是什么。思考题第 3 题让你回顾七层分析表，找到“分组对比”属于哪一层。

多维度快速扫描：让 AI 画散点矩阵

前面我们一次只看一个维度（票价分布、星级对比）。如果想一次看多个维度之间有没有"联动关系"，一对一对画太慢了。散点矩阵可以一次性把所有组合画出来。

什么时候用散点矩阵

- 刚拿到一份新数据集，想快速扫描全局
- 想知道"票价高的景区是不是销量也高？"
- 想发现你没想到的变量之间的关联

怎么向 AI 描述需求

【Prompt】
我有一份景区数据 df，包含 price（票价）、sale_count（销量）和 level（星级）三列。请用 Seaborn 画一个散点矩阵 (pairplot)，按 level 用不同颜色区分。

不需要记函数名和参数，把"看什么、按什么分"描述清楚就够了。

看到结果后关注什么

你看到的现象	意味着什么
两列的散点呈斜线	两个变量有关联（正相关/负相关）
散点均匀散开	两个变量没有明显关系
某个颜色的点聚成一团	该星级的特征和其他组不同
对角线直方图是偏斜的	该变量的分布不均匀（参考 §二 直方图）

核心价值：散点矩阵是"发现问题"的工具，不是"回答问题"的工具。看到有趣的模式后，再用箱线图或追溯代码深入分析。

【解释】

散点矩阵怎么用

这一页没有讲 API 参数（hue、diag_kind、height），因为你不需要记函数调用方式。你只需要知道三件事：什么时候用散点矩阵、怎么向 AI 描述需求、看到结果后关注什么。

右侧的"看到结果后关注什么"表格是这一页的核心——它教的是读图方法，不是代码实现。底部的"发现问题 vs 回答问题"的区分也很重要：散点矩阵是用来发现"哪两个变量值得深入看"的工具，不是用来下结论的。

实战发现：从数据到洞察的推理链

学会画图只是开始。真正的分析价值在于：从图表异常出发，追溯到具体数据，形成可解释的结论。

第一步：发现异常

在箱线图中，你可能发现 3A 景区组出现了票价超过 300 元的异常值——这不合常理，3A 景区通常票价较低。

第二步：追溯数据

```
# 找出 3A 景区中票价超过 300 元的记录
high_price_3a = filtered_df[
    (filtered_df['level'] == '3A') &
    (filtered_df['price'] > 300)
]
print(high_price_3a[['name', 'province', 'price']])
```

第三步：分析原因

追溯后可能发现：

- 某些"3A 景区"实际是包含了住宿/餐饮的综合度假区，票价是打包价
- 某些记录的票价字段被错误录入
- 个别景区近期涨价但星级评定未更新

第四步：形成结论

"3A 景区的高票价异常点主要来自综合度假区的打包价格，不代表纯观光景区的真实水平。在后续分析中应考虑单独标注或剔除此类记录。"

[解释]

从发现异常到得出结论

这一页展示了一个完整的"发现→追溯→分析→结论"链条。画图不是目的，从图中发现问题并给出可解释的结论才是。

代码示例展示了怎么用 DataFrame 条件筛选回到原始数据验证假设——这和之前学的 Pandas 操作是一样的。最后的结论模板展示了专业分析报告中的标准表述方式，到 L20 写研究报告时你会用到。

四、人机协同： Prompt 重心的转移

从格式调整转向业务逻辑描述

[解释]

为什么 Prompt 策略要变

上节课用 Matplotlib 画图时，需要在 Prompt 里交代很多格式细节（去边框、调颜色）。但 Seaborn 自带高质量的默认样式，格式方面的指令可以大幅省略。省下来的 Prompt 空间应该用来描述清洗逻辑和业务意图——这才是你真正需要负责的部分。

Prompt 的写法调整

Seaborn 内置了成熟的配色和排版方案，不需要在 Prompt 中花大量篇幅描述格式。重心应放在数据清洗逻辑和分析目标的描述上。

Prompt 的重心变化

- **格式方面可以省略：**多数大语言模型已经熟悉 Seaborn 的 API 和默认配置，不需要逐项指定颜色和边框。
- **把精力放在业务描述上：**在 Prompt 中明确告知需要过滤哪些数据、按什么字段分组、选择什么图表类型以及为什么选这种图。

Prompt 示例

【角色】数据分析工程师
【数据】我有一份旅游景区数据表 df，包含 price（票价）和 level（星级，如 5A/4A/3A）两列。
【任务】
1. 先过滤掉没有星级评定以及票价为零的记录；
2. 用过滤后的数据画一幅按星级分组的票价分布图；
3. 使用 Seaborn 的横向箱线图，方便对比不同星级的价格区间。

💡 **实操要点：**按照这种方式描述需求后，AI 会一次性生成完整的代码——包括前面的过滤语句、画布初始化和 Seaborn 函数调用。研究者只需要关注输出图表中的分布规律和异常值。

[解释]

Prompt 重心从格式转向逻辑

上节课用 Matplotlib 画图时，你需要在 Prompt 里告诉 AI 去掉边框、调整颜色等格式细节。换成 Seaborn 后，这些格式工作由库的默认设置自动处理了。所以你的 Prompt 应该把空间让给更有价值的内容：数据过滤条件（占掉什么）、分组逻辑（按什么分）和选用箱线图的原因。人负责定义分析目标和清洗逻辑，机器负责生成代码和渲染图表。

Prompt 踩坑实录：模糊指令导致错误输出

让 AI 画分布图时，指令描述的精确度直接决定输出质量。

失败示例

【Prompt】帮我画一个景区票价分布图。

AI 可能的输出：一张全国所有景区的价格直方图，包含了几万条零票价记录，图表右侧被几个极端高价压成了一条细线。

问题：没有指定过滤条件、分组方式和图表类型，AI 只能猜。

修正示例

【Prompt】
1. 从 df 中过滤掉 level 为"无"、price 为 0 的记录；
2. 用 Seaborn 画一幅按 level 分组的横向箱线图；
3. x 轴为 price, y 轴为 level。

AI 输出：一张干净的按星级分组箱线图，异常值清晰可见。

关键差异：过滤条件、分组字段、图表类型全部显式指定。

教训：越是复杂的分析任务，Prompt 越不能偷懒。"帮我画个图"是最差的 Prompt；"过滤什么、按什么分组、用什么图"才是合格的指令。

[解释]

为什么第一次 Prompt 会失败

对比两次 Prompt 的输出，能看出问题在哪。第一次失败是因为缺少三个关键信息：过滤条件（哪些数据要排除）、分组方式（按什么来分）、图表类型（画什么图）。修正版把这三项全部显式写出来了。这说明 AI 不会自己决定过滤什么数据——分析逻辑是你的责任。

向 AI 追问——票价越高，销量越高吗？

箱线图告诉我们票价分布的形状。但分布之外还有一个关键问题：票价和销量之间有没有关系？

Prompt

用 df 的 price 和 sale_count 字段，画一张散点图并叠加回归线。
过滤掉 price > 500 和 sale_count > 10000 的极端值，避免干扰趋势判断。

你可能会发现

- 票价和销量之间没有明显的正相关
- 很多高销量景区的票价反而很低（甚至免费）
- 回归线接近水平 → 票价不是决定销量的关键因素

AI 生成的散点图 + 回归线

(课堂现场由 AI 实时生成)

分析层次的升级

箱线图	散点图 + 回归线
② 分布分析	③ 关联分析
票价集中在哪个区间	票价和销量有没有关系

散点图能看出“低价高销”的趋势，但回归线可能被拉偏。
清洗阈值的选择直接影响回归方向——这就是为什么 Part 0.5 的离群点诊断如此重要。
教训：你的回归线方向，取决于你的清洗策略。

[解释]

从“集中在哪”到“有没有关系”

前面的箱线图回答的是“票价集中在哪个区间”（分布分析），这里进一步问“票价和销量有没有关系”（关联分析）。散点图+回归线是用来回答这种问题的经典工具。
注意：你能用肉眼看出“低价高销”的趋势，但回归线可能被极端值拉偏了。这正好说明 Part 0.5 的离群点诊断为什么重要——清洗策略直接影响你的统计结论。

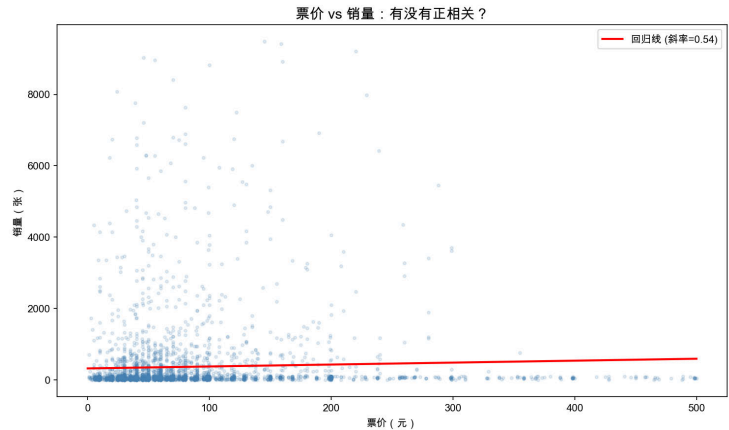
实操预期：票价与销量的关联分析

Notebook Part 3 预期结果

散点图 + 回归线揭示票价和销量之间的真实关系。

参考 Prompt

过滤 price 0-500、sale_count 0-10000，
画散点图 (X: price, Y: sale_count)，
叠加线性回归线，显示斜率值。



👉 思考题 3

1. 回归线的斜率是正还是负？接近零吗？
2. 高销量的点集中在什么价位区间？
3. 如果票价不是决定销量的关键因素，那什么可能是？

[解释]

视觉趋势和回归线为什么会不一致

散点图上你能用肉眼看出低价区间的点更密集、销量更高，这符合正常的供需规律。但回归线的方向可能被少数极端值拉偏了——几个黄金周暴涨的网红景区就能把整条线的斜率往上拽。这正好说明了 Part 0.5 为什么重要：清洗策略不同，回归线的方向就可能不同。如果你用箱线图确定更合理的清洗阈值，再重新画一次，结果可能会完全不同。

一张图扫描所有数值变量的关联强度

刚才只看了票价和销量两个变量。如果想同时看 price、rating、sale_count 之间的所有关系呢？

Prompt

用 df 的 price、rating、sale_count 三个数值字段，生成一张相关性热力矩阵，显示相关系数数值，配色用红蓝色阶。

AI 生成的相关性热力矩阵

（课堂现场由 AI 实时生成）

一次看清所有关系

用散点图一次只能看两个变量，相关性矩阵一张图就能扫描所有数值字段之间的关联强度。当你面对一份陌生的数据集时，**这是最快的全局扫描工具**。

怎么读这张图？

- 颜色越**红** → 正相关越强
- 颜色越**蓝** → 负相关越强
- 颜色接近**白色** → 基本无关联
- 对角线永远是 1.0（自己和自己的相关性）

[解释]

相关性矩阵什么时候用

这一页把分析工具从“两个变量的散点图”扩展到“多个变量的相关性矩阵”。你不需要记皮尔逊相关系数的数学原理，只需要知道怎么读图和什么时候用它。

典型场景：当你拿到一份陌生的数据集，不知道哪两个变量之间有关系时，先跑一张热力矩阵快速扫一遍。到 L20 写研究报告时，你选完题后第一步就可以用这张图来了解数据结构。

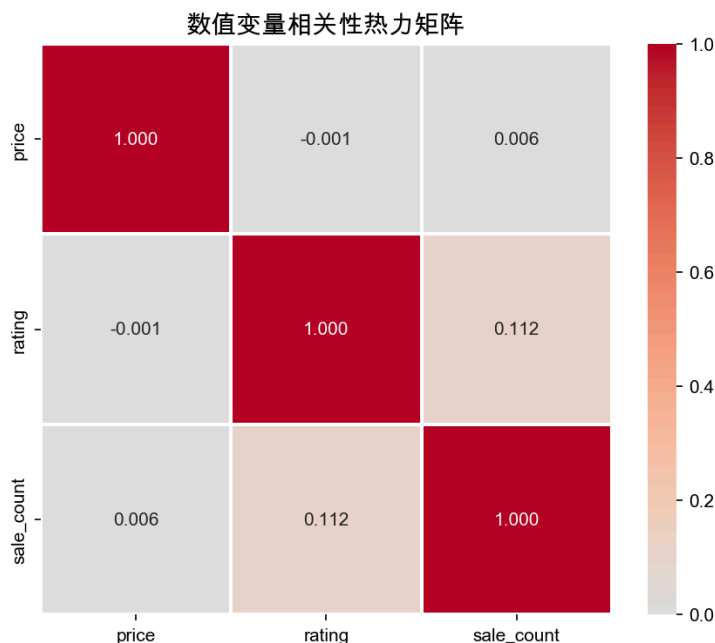
实操预期：相关性热力矩阵

Notebook Part 4 预期结果

一张图扫描所有数值变量之间的关联强度。

参考 Prompt

计算 price、rating、sale_count 的皮尔逊相关系数矩阵，用 `seaborn heatmap` 画出来，显示数值，配色用 `coolwarm`。



👉 思考题 4

1. 哪两个变量的相关性最强？正相关还是负相关？
2. rating 和 sale_count 的系数是否符合直觉？
3. 拿到一份陌生数据集，什么时候该先跑这张矩阵？

[解释]

什么时候该用热力矩阵

热力矩阵是一种“全局扫描”工具。当你拿到一份陌生的数据集时，不知道哪两个变量之间有关系，就可以先跑 `df.corr()` 配合 `heatmap` 快速扫一遍，找到值得深入研究的变量对，再用散点图单独分析。频色越红说明正相关越强，越蓝说明负相关越强，接近白色就是几乎没关系。

课堂实践：5 分钟快速验证

任务

选择你家乡的省份，用 Seaborn 画一张该省景区的票价直方图，观察分布形状。

参考代码

```
# 过滤出你家乡的省份数据
my = df[(df['province'] == '江西') &
        (df['price'] > 0)]

# 画直方图
sns.histplot(data=my, x='price', bins=20)
plt.title('江西省景区票价分布')
plt.xlabel('票价 (元)')
plt.show()
```

观察要点

完成后回答以下问题：

1. 票价分布是对称的还是偏斜的？
2. 大部分景区的票价集中在哪个区间？
3. 有没有明显"离群"的高价景区？

提交： 将直方图截图提交到课堂互动系统。把"江西"换成你自己的省份即可。

[解释]

为什么用家乡的数据画直方图

直方图 (histplot) 是本讲新学的工具，这里让你当场跑一遍。三个观察要点（偏斜方向、集中区间、离群值）对应本讲的核心概念。当你看到自己家乡的票价数据呈现出右偏分布时，你会实实在在理解为什么平均值会误导人。

本讲总结思考

分析层次回顾

1. 今天经历了 ②分布 → ⑤比较 → ③关联 三个层次，请用一句话总结每个层次的发现。
2. 散点图能看出“低价高销”的趋势，但回归线可能被拉偏。这说明了什么？

方法论反思

3. 离群点诊断（Part 0.5）对后续分析有什么影响？如果跳过会怎样？
4. 你在今天的实操中修改过几次 Prompt？最有效的修改是什么？

[解释]

回顾今天的分析链条

今天的操作链条是：离群点诊断 → 分布分析（直方图+箱线图）→ 分组比较（分组箱线图）→ 关联分析（散点图+回归线）→ 全局扫描（热力矩阵）。每一步都依赖前一步的结果——特别是最开头的离群点诊断，如果跳过它，后面所有图表的分布形状和回归方向都可能被极端值干扰。回头想想你今天调过几次 Prompt，最有效的修改是什么？

本讲总结与下讲预告

本讲完成了从总量排名到分布分析的方法升级，下一步将进入地理空间维度的数据可视化。

本讲知识线索

- 理解聚合统计（总数、平均值）的局限性：它们会掩盖组内的质量差异和极端值。
- 掌握离群点诊断流程：画图 → 诊断 → 清洗 → 再画图。
- 掌握 Seaborn 库的基本用法：散点图观察个体分布，箱线图对比分组差异。
- Prompt 策略调整：重心从格式描述转向清洗逻辑和业务意图。

下讲预告

下一讲我们将进入**地理空间可视化**。把景区的经纬度坐标投射到地图上，用标记点、热力图和等值线图揭示数据的空间分布规律。

💡 本模块的综合作业将在 **第 20 讲（模块收官）** 统一布置，届时需要把 L17-L19 学到的所有图表整合为一份完整的数据研究报告。

[解释]

本讲小结和下讲预告

这一页总结了今天的分析链条：聚合假象→离群点诊断→分布分析→关联分析→Prompt 策略调整。右边预告了下一讲的地理空间可视化。建议你把本讲产出的直方图、箱线图、散点图截图保存好，写研究报告时会直接用到。